

Fr. Conceicao Rodrigues College of Engineering

Father Agnel Ashram, Bandstand, Bandra-West, Mumbai – 400050

Department of Computer Engineering

Cloud Computing Lab: Mini-Project

IntelliPointer AI

A Screen-Aware AI Assistant on a Secure Cloud Backend

Under the guidance of

Dr. Vijay Shelke
Department of Computer Engineering

Submitted by

Ahamed Wafiq 10705
Sai Balkawade 10707
C. Jace Jaison 10711

Date of submission: 27 October 2026

Academic year: 2026–2027

Contents

Abstract **iii**

1 Introduction **1**
1.1 Problem statement 1
1.1.1 The problem 1
1.1.2 Why this needs a cloud solution 1
1.1.3 Scope 2
1.1.4 Stakeholders 2
1.2 Objectives 2

2 Project Description and Design **4**
2.1 Project planning 4
2.1.1 Approach and workflow 4
2.1.2 Work breakdown and deliverables 4
2.1.3 Risks and mitigations 5
2.1.4 Tools and technology 6
2.2 Cloud service selection 6
2.2.1 Selected services and why 6
2.2.2 Alternatives considered 7
2.2.3 Cost awareness 8
2.3 Architecture 8
2.3.1 Components 8
2.3.2 One question, end to end 9
2.3.3 Data model 9
2.3.4 Scalability and availability 10

3 Development and Security **11**
3.1 Implementation 11
3.1.1 Repository layout 11
3.1.2 Cloud API 11
3.1.3 Desktop application 12
3.1.4 Performance engineering 12

3.2 Security 13

4 Testing and Results **14**

4.1 Testing and results 14

4.1.1 Test strategy 14

4.1.2 Functional results (backend, real emulators) 14

4.1.3 Performance results 15

4.1.4 Deployment evidence (to be completed after deployment) 16

4.2 Register of experiments 16

4.2.1 Backend unit tests (13) 17

4.2.2 Backend integration tests on the real Firebase emulators (13) . . . 17

4.2.3 Deployment verification checks (12) 18

4.2.4 Desktop unit tests (3) 19

4.2.5 Desktop end-to-end suites (Electron) 20

4.2.6 Performance and design experiments (measured) 23

4.2.7 Security experiments and reviews 24

5 Conclusions and Future Work **26**

5.1 Limitations and future work 26

5.2 Conclusion 26

References **27**

Abstract

Asking an AI about something on the screen normally takes seven manual steps: take a screenshot, crop it, open a chatbot, paste the image, retype the context, wait, and switch back. IntelliPointer AI reduces this to one shortcut. A Windows application freezes the screen, lets the user mark one or more regions, reads the text on the PC, and shows a streamed answer from a vision-capable model in a small floating window, with follow-up questions that reuse the same screenshot.

A model needs a secret key, and a key inside a desktop program can be extracted. The project therefore places a stateless HTTPS API in the cloud (Render or Google Cloud Run). Firebase Authentication identifies each user with short-lived signed tokens, and Cloud Firestore holds a per-user daily quota that is updated in an atomic transaction, so concurrent requests cannot exceed it. The model key never leaves the server, nothing the user captures or asks is stored or logged, and every control is covered by an automated test, including integration tests against the real Firebase emulators. On the test machine the overlay appears in under 40 ms and a finished crop is ready in about 0.2 to 0.5 s.

Keywords: cloud computing, Firebase, Firestore, serverless, Server-Sent Events, authentication, quota, desktop application, vision model.

List of Figures

Figure 2.1 System architecture. Numbered flows: 0 sign in, 1 request, 2 streamed response, 3 token verification, 4 quota transaction, 5 model call. 8
Figure 2.2 Sequence of one question: sign in, local capture and OCR, authenticated request, quota transaction, streamed answer. 10

List of Tables

Table 1.1 Stakeholders 2
Table 1.2 Objectives 2

Table 2.1 Work breakdown and deliverables 4
Table 2.2 Risks and mitigations 5
Table 2.3 Selected services and why 6
Table 2.4 Alternatives considered 7
Table 2.5 Data model 9

Table 3.1 Cloud API 11
Table 3.2 Desktop application 12
Table 3.3 Performance engineering 12
Table 3.4 Security 13

Table 4.1 Test strategy 14
Table 4.2 Functional results (backend, real emulators) 15
Table 4.3 Performance results 15
Table 4.4 Deployment evidence (to be completed after deployment) 16
Table 4.5 Backend unit tests (13) 17
Table 4.6 Backend integration tests on the real Firebase emulators (13) 18
Table 4.7 Deployment verification checks (12) 19
Table 4.8 Desktop unit tests (3) 19
Table 4.9 Selection overlay (test/overlay.e2e.js, 10 checks) 20
Table 4.10 Panel, model menu, settings and commands (test/ui.e2e.js, 16 checks) 20
Table 4.11 Selected text and clipboard (test/selection.e2e.js, 10 checks) 21
Table 4.12 Screen-region shortcut speed (test/capture.e2e.js, 3 rounds, one of them after 15 s idle) 21
Table 4.13 Overlay excluded from screenshots (test/capture-exclusion.e2e.js, 2 checks) 22
Table 4.14 Model providers and key storage (test/providers.e2e.js) 22
Table 4.15 Live conversation through the cloud API (test/panel.e2e.js) 23
Table 4.16 Performance and design experiments (measured) 23
Table 4.17 Security experiments and reviews 24

Chapter 1

Introduction

1.1 Problem statement

1.1.1 The problem

Getting an AI answer about something on screen is slow and breaks concentration. The usual routine is:

1. take a screenshot,
2. crop it,
3. open a chatbot website,
4. paste the image,
5. retype the context,
6. wait, then
7. switch back to the original application.

For a student reading a paper, an engineer debugging an error dialog, or anyone comparing two charts, this is seven manual steps for one question.

1.1.2 Why this needs a cloud solution

Putting the AI call directly in a desktop program creates three problems that only a cloud design solves:

- **Secrets.** Model APIs need keys. A key inside a distributed program can be extracted, and then anyone can spend the owner's money.
- **Identity and fairness.** Without a server there is no way to know who is calling or to limit how much each person can use, so one user (or one script) can exhaust the quota for everyone.
- **Elasticity and operations.** Model traffic is bursty. A managed, stateless service can scale with demand, and it gives one place to monitor latency and failures.

1.1.3 Scope

In scope. A Windows desktop application (select a screen region or text, ask, get a streamed answer in a floating panel) and a cloud API that authenticates users, enforces per-user quotas, validates input, routes to an AI model with vision, and streams the answer back.

Out of scope. A mobile or web client, training models, storing conversations or screenshots in the cloud (deliberately excluded for privacy), and billing.

1.1.4 Stakeholders

Table 1.1. Stakeholders

Stakeholder	Need
End user	One shortcut from seeing something to an answer, with clear control over what is sent
Service owner	No leaked keys, bounded cost, visibility into usage and failures
Evaluator	Clear architecture, working system, evidence of security and testing

1.2 Objectives

Each objective is measurable and is checked in §8.

Table 1.2. Objectives

#	Objective	Success measure
O1	Shortcut → selection overlay → finished crop, fast	Overlay on screen in under 100 ms; crop ready in under 1 s (§8.3)
O2	Mark exactly what to ask about	Rectangle, ellipse, lasso, pen, arrow, highlight and blur tools; several regions per question
O3	Understand text and images	Local OCR plus a vision-capable model; follow-up questions reuse the screenshot
O4	Stream answers into a floating panel	First tokens render as they arrive; stop, retry and copy work
O5	Secure, authenticated cloud API	Every request needs a verified identity; limits and validation enforced
O6	Per-user usage control in the cloud	Quota never exceeded, even under concurrent requests

#	Objective	Success measure
O7	Privacy by design	No screenshots or prompts stored or logged; the user is told what is sent
O8	Measurable performance	Capture, OCR, upload, first-token and total latency recorded
O9	Lightweight client	Low memory and idle CPU
O10	Reproducible deployment and tests	One-command verification of a deployed service; automated test suites

Chapter 2

Project Description and Design

2.1 Project planning

2.1.1 Approach and workflow

The system was built in vertical slices, each ending in something testable, rather than layer by layer:

1. **Desktop shell and capture** (global shortcut, transparent overlay, frozen screenshot).
2. **Annotation tools and cropping** (shapes, masking, blur; crops at physical resolution under any display scaling).
3. **Local processing** (OCR, thumbnails, a "what will be sent" notice).
4. **AI integration** (provider abstraction, streaming, follow-ups).
5. **Cloud backend** (authentication, quotas, validation, provider routing, SSE).
6. **Cloud deployment and verification** (Firebase, Render or Cloud Run, verification script).
7. **Hardening and performance** (security review, dependency audit, latency and memory work).
8. **Documentation and evaluation** (architecture, security and deployment documents, this report).

2.1.2 Work breakdown and deliverables

Table 2.1. Work breakdown and deliverables

Phase	Work	Deliverable	Verified by
1	Electron shell, global shortcuts, tray	App starts in the tray; shortcut opens the overlay	Electron tests
2	Drawing tools, regions, masks, blur	Overlay with all tools	Overlay test at 1.5× scaling

Phase	Work	Deliverable	Verified by
3	Crops, OCR, privacy notice	Panel shows thumbnails, OCR text, what is sent	UI test
4	Providers, streaming, follow-ups	Streamed answers, retry, stop, follow-up	Live test with a real model
5	Express API, Firebase Auth, Firestore quotas	`backend/`	13 unit and 13 emulator tests
6	Render / Cloud Run deployment, verifier	`render.yaml`, `Dockerfile`, `verify-deployment.mjs`	Verifier output (§8.4)
7	Security review, audit, performance	`docs/SECURITY.md`, 0 vulnerabilities, latency table	§7, §8
8	Documentation	README files, guides, this report	Review

2.1.3 Risks and mitigations

Table 2.2. Risks and mitigations

Risk	Likeliho	Impact	Mitigation	Outcome
Model provider is slow or overloaded	High	User-visible delay	Streaming, clear loading and error states, retry, a fast model by default, a speed test per model	Observed: 3 s to 20 s time-to-first-token depending on provider load (§8.3)
Leaked provider key	Medium	Cost, abuse	Keys only in server secrets; none in the client; no key file on Cloud Run	No secret in the repository (checked before every commit)
Cost or abuse by a single user	Medium	Quota exhaustion	Auth + per-user daily quota + rate limits	Verified under a 12-request burst (§8.2)
Capture differs across Windows display scaling	Medium	Wrong crops	Capture in physical pixels; tested at 1.5× scaling	Test passes
Free-tier cold starts	High	First request is slow	Documented; health check endpoint; Cloud Run `min-instances` option	Measured by the verifier

Risk	Likeliho	Impact	Mitigation	Outcome
Cloud account unavailable (billing)	Medium	Cannot deploy to Google Cloud	Two targets: Render (free) and Cloud Run	Both documented

2.1.4 Tools and technology

Electron (desktop shell), plain JavaScript (no build step, fast start), a small C# native helper for Windows APIs, Node.js 22 with Express, Firebase Authentication and Cloud Firestore, Google Gemini API, Render or Google Cloud Run, Docker, GitHub Actions (CI), tesseract.js (local OCR), Lucide (icons).

2.2 Cloud service selection

2.2.1 Selected services and why

Table 2.3. Selected services and why

Need	Service	Why this one
User identity	**Firebase Authentication**	Managed sign-in and short-lived signed ID tokens (JWT, one hour) that the API verifies with Google's public keys. No password database to build or protect. Supports revocation and disabling users.
Per-user usage and quotas	**Cloud Firestore**	A serverless document database with **transactions**, so "check the quota and add one" is atomic and concurrent requests cannot exceed the limit. Scales without servers to manage. Only counters are stored.
Running the API	**Render web service** (free) or **Google Cloud Run**	Managed HTTPS and certificates, deploy from Git or a container, secret environment variables, health checks. Cloud Run additionally scales instances automatically, supports least-privilege service accounts, and removes the need for any key file.

Need	Service	Why this one
AI inference	**Google Gemini API** (optional: any OpenAI-compatible service)	Vision input, token streaming and web grounding in one API. Called only from the backend, so the key never reaches users.
Secrets	Host secret environment variables; **Secret Manager** on Cloud Run	The only place a key exists.
Observability	Platform logs and structured JSON logs	One line per request with request id, user id, model, outcome and latencies. No content.

2.2.2 Alternatives considered

Table 2.4. Alternatives considered

Decision	Alternative	Why not chosen
Identity	Build our own login (database + password hashing + session tokens)	More code and more attack surface for no benefit; easy to get wrong
Identity	AWS Cognito / Azure AD B2C	Comparable, but Firebase integrates directly with Firestore rules and Google tooling, and has a free tier suited to a student project
Quota store	A relational database	A heavier service to run for a handful of counters; Firestore transactions give exactly the atomicity needed
Quota store	In-memory counters	Lost on restart and not shared between instances (kept only as a local-development fallback)
Conversation storage	Store chats in Firestore	Rejected on privacy grounds: it would mean storing screenshots and prompts. The client keeps conversations in memory only
Compute	Virtual machine	More to patch and secure, no automatic scaling or managed TLS
Compute	Cloud Functions	Long-lived streaming responses fit a container service better

Decision	Alternative	Why not chosen
AI	Calling the model from the desktop app	Would put the provider key in the client

We did not add services for their own sake: there is no message queue, cache or extra database because no requirement needs one.

2.2.3 Cost awareness

All chosen services have free tiers sufficient for development and a demonstration (check the current limits on the providers' pricing pages before relying on them). Cost is also *bounded by design*: the daily quota caps requests per user, request size is limited, upstream calls time out, and Cloud Run offers `--max-instances` as a hard ceiling.

2.3 Architecture

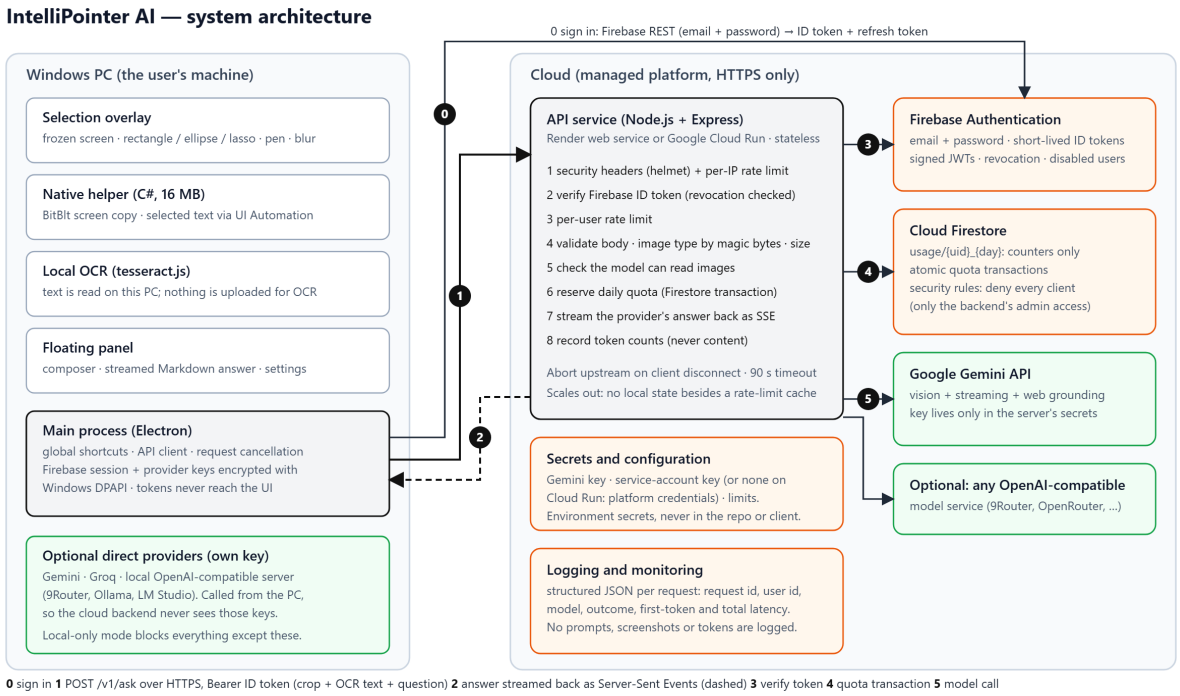


Figure 2.1. System architecture. Numbered flows: 0 sign in, 1 request, 2 streamed response, 3 token verification, 4 quota transaction, 5 model call.

2.3.1 Components

Windows client (Electron).

- The *main process* owns the shortcuts, the API client, request cancellation, and the

encrypted Firebase session and provider keys (Windows DPAPI).

- *Overlay windows* (one per display) draw the selection tools over a frozen screenshot.
- The *panel* shows the conversation, the model menu and settings.
- A 16 MB *native helper* provides fast screen copies and reads selected text.
- *Local OCR* runs on the PC.
- Renderers are sandboxed and see no tokens or keys.

API service (Node.js + Express), stateless. Per request: security headers and a per-IP rate limit → verify the Firebase ID token (with revocation check) → per-user rate limit → validate the body and images (file signature, size, count) → check the model can read images → reserve the daily quota in a Firestore transaction → stream the provider's answer as Server-Sent Events → record token counts → log the outcome.

Firebase Authentication. Issues and verifies identity tokens.

Cloud Firestore. One small document per user per day, `usage/{uid}_{YYYY-MM-DD}`, holding only counters. Security rules deny every client; only the backend's admin access reads or writes.

AI provider. Gemini through its official streaming API, behind a provider registry that also accepts any OpenAI-compatible endpoint.

2.3.2 One question, end to end

2.3.3 Data model

Table 2.5. Data model

Store	Content	Retention
Firestore `usage/{uid}_{day}`	`uid`, `day`, `requests`, `inputTokens`, `outputTokens`, `lastProvider`, `lastModel`, `updatedAt`	Per day; no content
Desktop `%APPDATA%`	Non-secret settings; DPAPI-encrypted session and provider keys; a timings-only CSV and diagnostics log	Until the user clears it
Anywhere	Screenshots, OCR text, prompts, answers	**Never stored**; in memory until the conversation is cleared

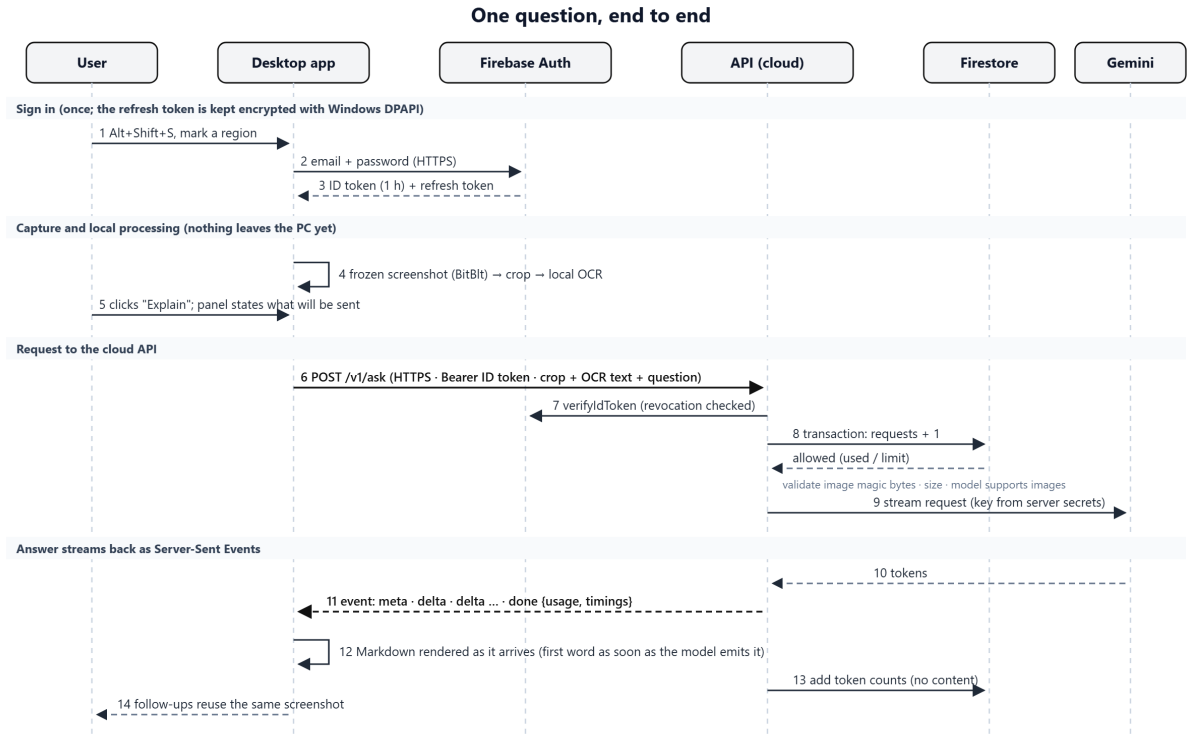


Figure 2.2. Sequence of one question: sign in, local capture and OCR, authenticated request, quota transaction, streamed answer.

2.3.4 Scalability and availability

- **Horizontal scaling.** The API keeps no session state (identity is in the token, quota in Firestore), so any number of instances can serve any request. Cloud Run adds instances automatically with load.
- **Atomic shared state.** The quota is a Firestore transaction on one small document per user-day, so instances never disagree. A single document sustains roughly one write per second, which is far above one user's request rate.
- **Streaming.** Responses are Server-Sent Events, so the user sees text as soon as the model produces it; the server aborts the upstream call if the client leaves.
- **Failure handling.** Upstream errors become a retryable SSE error event; the client can retry or switch model; health checks let the platform restart a failing instance; a 90-second upstream timeout prevents hung requests.
- **Known scaling limit.** The per-minute rate limiter is per instance (in memory). The daily quota, which bounds cost, is global. A shared limiter is the next step for large deployments (§9).

Chapter 3

Development and Security

3.1 Implementation

3.1.1 Repository layout

```
backend/    Express API, tests, emulator integration test, deployment verifier,
    Dockerfile
desktop/    Electron app (main process, renderers, native helper, installer scripts,
    tests)
firebase/   Firestore security rules      firebase.json   Emulator configuration
docs/       Architecture, security and deployment documents, this report
render.yaml Render blueprint              .github/workflows/ci.yml   Continuous
    integration
```

3.1.2 Cloud API

Table 3.1. Cloud API

Endpoint	Auth	Purpose
`GET /healthz`	none	Liveness; whether providers and Firestore are configured
`GET /v1/models`	Bearer	Models with capability flags (vision, web search)
`GET /v1/usage`	Bearer	The caller's usage and limit for today
`POST /v1/ask`	Bearer	Streams SSE events `meta`, `delta`, `sources`, `done`, `error`

Key design points: Firebase tokens are verified on every request with revocation checking; the daily quota is reserved in a transaction before any model is called; images are validated by file signature; the provider layer is an abstraction (`gemini`, OpenAI-compatible) with per-model capability checks; logs contain identifiers, counts and timings only.

3.1.3 Desktop application

Table 3.2. Desktop application

Feature	How
Overlay appears instantly	Pre-created transparent overlays are shown at once; the screenshot is taken behind them (they are excluded from capture) and swapped in as a frozen background
Correct under any display scaling	Capture and crops in physical pixels; tested at 1.5× scaling
Selection tools	Rectangle, square, ellipse, lasso, pen, arrow, highlight, blur; several regions per question; undo and redo; mask outside shapes
Selected-text questions	A native helper reads the selection through UI Automation, with a Ctrl+C fallback that restores the clipboard
Commands and shortcuts	Built-in and user-defined commands, optional per-command shortcuts, pinned to the toolbars; a recorder that checks shortcuts against other applications
Model sources	Gemini or Groq with the user's own key, any OpenAI-compatible server (9Router, Ollama, LM Studio), or this project's cloud API; a model menu with image support and measured speed
Privacy	A notice before every send, a blur tool, a local-only mode, nothing written to disk

3.1.4 Performance engineering

Measured on a 1920×1080 display at 125 % scaling.

Table 3.3. Performance engineering

Change	Effect
Show the overlay first, capture behind it	Overlay on screen in 5–39 ms instead of waiting for the capture
Screen copy through GDI BitBlt in a native helper	Screenshot in 109–338 ms; Electron's built-in grabber took up to 2.8 s after the app had been idle
Compiled C# helper instead of a PowerShell host	16 MB instead of about 127 MB; ready in about 100–400 ms
Software rendering by default	About 100 MB less memory with no visible difference for this interface

Change	Effect
OCR engine loaded on demand and unloaded after idle	No cost until a capture is started

3.2 Security

The full control-by-control analysis, with the code location and the test that proves each control, is in `docs/SECURITY.md`. Summary:

Table 3.4. Security

Area	Controls
Authentication	Firebase ID token required for every `/v1/*` call; signature and revocation verified; disabled and revoked users refused immediately
Authorization	Quota and usage keyed by the verified user id; Firestore rules deny all client access, including a user reading or resetting their own quota
Secrets	Gemini key and service account only in server secrets; none in the repository or the client; no key file at all on Cloud Run; user keys encrypted with Windows DPAPI
Input validation	Size, count and length limits; images checked by file signature (a fake PNG is rejected); a model that cannot read images never receives one
Abuse and cost control	Per-IP and per-user rate limits; daily quota in an atomic transaction; upstream timeout; upstream cancelled when the client disconnects
Transport and platform	HTTPS only (the client refuses non-HTTPS backends except localhost); `helmet` security headers; non-root container; 0 known vulnerabilities in production dependencies, enforced in CI
Privacy	Nothing stored or logged: no screenshots, OCR text, prompts or tokens; users are told what will be sent; a blur tool and local-only mode
Client hardening	Sandboxed renderers, context isolation, strict content-security policy, tokens never reach the UI
Safe development mode	The authentication bypass used for local work is refused in production or on a public interface

A STRIDE threat model and an honest list of remaining limitations (per-instance minute limiter, no App Check, no email verification, unsigned installer) are in `docs/SECURITY.md`.

Chapter 4

Testing and Results

4.1 Testing and results

4.1.1 Test strategy

Table 4.1. Test strategy

Level	What	Command	Count
Backend unit	Validation (magic bytes, limits, roles), authentication enforcement, security headers	`npm test`	13
Backend integration (real Firebase emulators)	Real token verification, Firestore transactions and security rules, SSE streaming, rate limits, cancellation, log redaction	`npm run test: emulator`	13
Deployment verification	The same kind of checks against a live URL	`npm run verify: deployment`	12 checks
Desktop unit	SSE parser, compiled helper (startup, screen grab, memory)	`npm test`	3
Desktop UI and end to end	Overlay tools and crops, panel and settings, selected text and clipboard, capture speed, a live model conversation, providers	`npx electron test/*.e2e. js`	6 suites
Static	Syntax check of every file; dependency audit	`npm run check`, `npm audit`	41 files; 0 vulnera- bilities
Continuous integration	All of the above on every push	GitHub Actions	

4.1.2 Functional results (backend, real emulators)

Table 4.2. Functional results (backend, real emulators)

Property tested	Result
Requests without a valid Firebase token are refused	Passed (401)
A real emulator-issued token is accepted; models listed with capabilities	Passed
`/v1/ask` streams `meta, delta..., done` with token usage, and records usage in Firestore	Passed; only counters are stored
Quota under concurrency: 12 parallel requests with a limit of 5	**Exactly 5 allowed and 7 refused; the counter never overshoots**
One user's quota does not affect another's	Passed
Revoked sessions and disabled accounts are refused at once	Passed
Image on a text-only model, oversized image, unknown model	Refused with the right errors
Upstream failure	Retryable SSE error event
Client disconnects	The upstream request is cancelled
Per-user rate limit	429 after the limit, independent of the daily quota
Logs never contain prompts or tokens	Passed
Firestore rules deny every client read and write	Passed, including a user's own document

4.1.3 Performance results

Table 4.3. Performance results

Measurement	Result
Shortcut → overlay on screen	5–39 ms
Shortcut → screenshot frozen	109–338 ms
Shortcut → finished crop (including a cold round after 15 s idle)	228–519 ms across all measured runs
Local OCR of a small code snippet	0.7–1.3 s
Upload size for a crop, OCR text and question	about 3 KB

Measurement	Result
Backend time before the model call (auth, validation, quota)	about 1 ms locally with in-memory quota; *fill in with Firestore from the deployed service*
Time to first token (varies a great deal with provider load)	0.7 s with a lightweight Gemini model through the backend in one run; 3 s to 20 s in other runs, including a period when the provider returned "high demand" errors, which the app shows as a retryable error
Desktop app memory at idle (private)	about 183 MB in total (app about 167 MB, native helper 16 MB); before the optimisations about 397 MB (app about 270 MB with GPU acceleration, PowerShell helper about 127 MB)
Desktop idle CPU	0.1–0.3 %

Interpretation: the client-side work (capture, OCR, upload) takes well under a second; almost all waiting is model inference at the provider, which varies with load. Streaming and a per-stage timing line make that visible instead of hiding it.

4.1.4 Deployment evidence (to be completed after deployment)

Run `npm run verify:deployment` (see `docs/DEPLOYMENT.md`), then paste the generated table and add the screenshots:

Table 4.4. Deployment evidence (to be completed after deployment)

Item	Source
Verifier table (HTTPS, health, security headers, 401 checks, sign-in, streamed answer with latency, usage counter, 415 and 413 checks)	`docs/evidence/deployment-<time>.md`
Firebase Authentication → Users	*screenshot*
Firestore → `usage` documents (counters only)	*screenshot*
Render or Cloud Run dashboard and request logs	*screenshot*
The desktop app answering through the deployed backend	*screenshot*

4.2 Register of experiments

Every experiment, test and measurement carried out on IntelliPointer AI, by name. "Pass" means the automated check passed in the final run (`npm run test:all` in `backend/`, `npm`

`test` and the `test/*.e2e.js` suites in `desktop/`). Measured values are from a 1920×1080 display at 125 % scaling.

4.2.1 Backend unit tests (13)

Command: `cd backend && npm test`

Table 4.5. Backend unit tests (13)

ID	Experiment	What it shows	Result
B1	Accepts a text question	A valid request is parsed	Pass
B2	Accepts a real PNG	A genuine image passes validation	Pass
B3	Rejects a declared PNG whose bytes are not PNG	File type is checked by signature, not by name	Pass
B4	Rejects an unsupported MIME type	Only PNG, JPEG and WebP are allowed	Pass
B5	Rejects images in assistant turns and bad roles	Message structure is enforced	Pass
B6	Last message must be from the user	Malformed conversations are refused	Pass
B7	Rejects oversized text	Length limit enforced	Pass
B8	Recognises JPEG and WebP signatures	Image sniffing works for all allowed types	Pass
B9	Health endpoint is public	Platform health checks work without login	Pass
B10	Ask without a token is 401	Authentication is required	Pass
B11	Ask with a forged token is 401	Fake tokens are refused	Pass
B12	Models endpoint requires authentication	No anonymous access	Pass
B13	Security headers are set and `x-powered-by` is hidden	Basic hardening in place	Pass

4.2.2 Backend integration tests on the real Firebase emulators (13)

Command: `cd backend && npm run test:emulator` (real Firebase Authentication and Cloud Firestore emulators; a local fake model server).

Table 4.6. Backend integration tests on the real Firebase emulators (13)

ID	Experiment	What it shows	Result
E1	Health reports Firestore-backed quotas	The service is wired to Firestore	Pass
E2	Requests without a valid Firebase ID token are rejected	Authentication enforced with real tokens	Pass
E3	A real emulator ID token is accepted and models are listed with capabilities	Token verification and the model list	Pass
E4	`/v1/ask` streams SSE (meta, delta, done) and records usage in Firestore	Streaming works; only counters are stored	Pass
E5	Daily quota holds under a burst of concurrent requests	12 parallel requests, limit 5: exactly 5 allowed, counter never overshoots (atomic transactions)	Pass
E6	One user's quota does not affect another user's	Per-user isolation	Pass
E7	Revoked sessions and disabled accounts are rejected	Immediate access removal	Pass
E8	Input validation: image size, image on a text-only model, unknown model	Correct refusals (413, 400)	Pass
E9	Upstream failures surface as a retryable SSE error	Graceful failure handling	Pass
E10	A client that disconnects cancels the upstream request	No wasted model calls	Pass
E11	Logs never contain prompts or tokens	Privacy of logs	Pass
E12	Per-user rate limit returns 429 independent of the daily quota	Abuse protection	Pass
E13	Firestore security rules deny every client read and write	A user cannot read or reset their own quota	Pass

4.2.3 Deployment verification checks (12)

Command: `cd backend && npm run verify:deployment -- --url <service URL> . . .` (run locally against the real Gemini API: 12 of 12; to be run again on the deployed service).

Table 4.7. Deployment verification checks (12)

ID	Check	Result
V1	HTTPS is used	Pass (local)
V2	`GET /healthz` responds	Pass
V3	Warm response time	Pass
V4	Quotas are stored in Firestore	Pass locally (in-memory expected); must show `firestore` when deployed
V5	Security headers present	Pass
V6	`GET /v1/models` lists models with capability flags	Pass
V7	`POST /v1/ask` streams an answer	Pass (first token 0.7–3.6 s in two runs)
V8	Answer is delivered as Server-Sent Events	Pass
V9	Usage counter incremented	Pass
V10	An image question is accepted by a vision model	Pass
V11	A file that is not really a PNG is rejected (415)	Pass
V12	An oversized request is rejected (413)	Pass

4.2.4 Desktop unit tests (3)

Command: `cd desktop && npm test`

Table 4.8. Desktop unit tests (3)

ID	Experiment	What it shows	Result
D1	SSE parser handles events split across chunks	Streaming parser is robust	Pass
D2	SSE parser defaults the event name and ignores comments	Protocol edge cases	Pass
D3	Compiled native helper starts, grabs the screen and stays small	Ready in about 100 ms; 1080p grab about 110 ms; 16 MB	Pass

4.2.5 Desktop end-to-end suites (Electron)

Selection overlay (test/overlay.e2e.js, 10 checks)

Table 4.9. Selection overlay (test/overlay.e2e.js, 10 checks)

ID	Experiment	Result
O1	Overlay is active before the screenshot arrives	Pass
O2	User can start selecting immediately	Pass
O3	Screenshot swapped in as the frozen background	Pass
O4	Enter confirms the selection	Pass
O5	Two regions captured	Pass
O6	Region shapes are rectangle and ellipse	Pass
O7	A region can be moved (200, 210)	Pass
O8	Region cropped at physical resolution (449×299 at 1.5× scaling)	Pass
O9	Crop is a PNG	Pass
O10	No renderer console errors	Pass

Panel, model menu, settings and commands (test/ui.e2e.js, 16 checks)

Table 4.10. Panel, model menu, settings and commands (test/ui.e2e.js, 16 checks)

ID	Experiment	Result
U1	Ask bar sizes itself to its commands	Pass
U2	Custom command in the Ask bar sends its id	Pass
U3	Empty state shows the new shortcuts	Pass
U4	One button per command	Pass
U5	Model button shows the current model	Pass
U6	Models are grouped by source	Pass
U7	Search filters the model list	Pass
U8	Enter chooses the highlighted model	Pass
U9	Menu closes after choosing	Pass
U10	Source list shows Gemini, Groq, 9Router and IntelliPointer Cloud	Pass
U11	Each source has a show-in-menu switch	Pass

ID	Experiment	Result
U12	Command editor opens as a sheet	Pass
U13	New command added to the list	Pass
U14	Saved command persisted to settings	Pass
U15	Shortcut recorder shows Alt+Shift+S	Pass
U16	No renderer console errors	Pass

Selected text and clipboard (test/selection.e2e.js, 10 checks)

Table 4.11. Selected text and clipboard (test/selection.e2e.js, 10 checks)

ID	Experiment	Result
S1	Helper became ready	Pass
S2	Selected text read via UI Automation	Pass
S3	Automation read is fast when the helper answers	Pass
S4	Concurrent requests both resolve	Pass
S5	Ctrl+C fallback is skipped for consoles	Pass
S6	The user's text clipboard is restored after the copy attempt	Pass
S7	Clipboard image restored and read back as PNG	Pass
S8	Text and HTML restored together	Pass
S9	Recent mouse selection remembered for fallback	Pass
S10	Stale mouse selections are ignored	Pass

Screen-region shortcut speed (test/capture.e2e.js, 3 rounds, one of them after 15 s idle)

Table 4.12. Screen-region shortcut speed (test/capture.e2e.js, 3 rounds, one of them after 15 s idle)

ID	Experiment (per round)	Result
C1	Overlay painted on screen in under 100 ms	5–39 ms
C2	Confirm before the screenshot arrives still returns a full-resolution crop	Pass
C3	Fast BitBlt path used	Pass

ID	Experiment (per round)	Result
C4	Screenshot ready in under 600 ms	109–338 ms
C5	Shortcut to finished crop in under 1 s	228–519 ms
C6	Overlay hidden after confirm	Pass
C7	Overlay reset (no stale frame next time)	Pass

Overlay excluded from screenshots (test/capture-exclusion.e2e.js, 2 checks)

Table 4.13. Overlay excluded from screenshots (test/capture-exclusion.e2e.js, 2 checks)

ID	Experiment	Result
X1	A content-protected window is excluded from the capture	Pass
X2	A normal window is captured (control)	Pass

Model providers and key storage (test/providers.e2e.js)

Table 4.14. Model providers and key storage (test/providers.e2e.js)

ID	Experiment	Result
P1	Plain-HTTP local server outside the local network is rejected	Pass
P2	LAN HTTP URL accepted and normalised	Pass
P3	API keys are not stored in plain text	Pass
P4	The interface only sees the last four characters of a key	Pass
P5	Gemini: listed chat models	Pass (20 models)
P6	Gemini: chosen model appears in the model menu	Pass
P7	Gemini: streamed a real answer directly	Pass
P8	Local server: embedding models filtered out	Pass
P9	Local server: image support guessed from model names	Pass
P10	Local server: model listed under its display name	Pass
P11	Local server: streamed answer assembled from SSE chunks	Pass
P12	Local server: optional API key sent as a Bearer token	Pass

ID	Experiment	Result
P13	Local server: image sent in the OpenAI image format	Pass
P14	An invalid Gemini key produces a readable error	Pass
P15	Groq: listing, streaming (needs a Groq key)	Not run (no key available)

Live conversation through the cloud API (test/panel.e2e.js)

Table 4.15. Live conversation through the cloud API (test/panel.e2e.js)

ID	Experiment	Result
L1	Panel loaded models from the backend	Pass
L2	Local OCR read the code text from the screenshot	Pass (0.7–1.3 s)
L3	Privacy notice lists what will be sent	Pass
L4	Answer streamed without error	Pass (first token 3.2–22 s depending on provider load)
L5	Answer contains the fixed code	Pass
L6	Follow-up answered from the same screenshot context	Pass
L7	No renderer console errors	Pass

4.2.6 Performance and design experiments (measured)

Table 4.16. Performance and design experiments (measured)

ID	Experiment	Compared	Result
F1	Screenshot method	Electron's grabber vs GDI BitBlt	2.4–2.8 s after idle vs 109–338 ms
F2	Overlay ordering	Capture first, then show vs show first, capture behind	250 ms to over 2.5 s vs 5–39 ms to appear
F3	Native helper host	PowerShell host vs compiled C# program	About 127 MB and 1.6–4.6 s to start vs 16 MB and 100–400 ms

ID	Experiment	Compared	Result
F4	GPU acceleration	On vs off (software rendering)	GPU process 128 MB vs 25 MB; total private memory 270 MB vs 164 MB; crop time unchanged
F5	App memory overall	Before vs after the optimisations	About 397 MB to about 183 MB
F6	Idle CPU	After optimisation	0.1–0.3 %
F7	OCR engine lifecycle	Always loaded vs on demand and unloaded when idle	No cost until a capture starts
F8	Model latency	Gemini through the backend, repeated runs	0.7 s to about 20 s to first token; provider overload produced HTTP 503 and a 90 s timeout, shown as a retryable error
F9	Model speed test	9Router models listed with vision flags (70 of 105 read images)	Fastest chosen per measured first-token time
F10	Clean-clone deployment test	Fresh checkout, `npm ci --omit=dev`, production start	Health OK, unauthenticated request 401

4.2.7 Security experiments and reviews

Table 4.17. Security experiments and reviews

ID	Experiment	Result
G1	Dependency audit, backend (before)	8 moderate findings, all from one transitive package
G2	Dependency audit after upgrading firebase-admin	0 vulnerabilities (backend and desktop)
G3	Revocation timing experiment	Revocation has one-second resolution; test waits out the second
G4	Firestore rules experiment	Anonymous and signed-in clients both refused (403); admin access allowed
G5	Secret scan before every commit	No key or token in the repository or its history
G6	STRIDE threat model	Documented in `docs/SECURITY.md`

ID	Experiment	Result
G7	Key-leak response drill	Procedure documented in `docs/DEPLOYMENT.md`

Chapter 5

Conclusions and Future Work

5.1 Limitations and future work

- The per-minute rate limit is per instance; a shared limiter would be needed at large scale.
- Firebase App Check, email verification and sign-in with Google are not enabled.
- The free Render tier sleeps when idle, so the first request after a pause is slow; Cloud Run with `--min-instances 1` avoids this.
- The Windows installer is not code-signed.
- Model latency is outside our control; a lighter model or a nearer region helps.
- Future: shared rate limiting, an admin dashboard for usage, more providers through the existing registry, Windows' built-in OCR engine.

5.2 Conclusion

IntelliPointer AI reduces "see something, get an answer" to one shortcut, one drag and one click, and shows the answer in a floating window without switching applications. The cloud side demonstrates a secure design built from managed services: Firebase Authentication for identity, Firestore transactions for a quota that cannot be exceeded even under concurrent load, a stateless HTTPS API on Render or Cloud Run that scales out, and secrets that never leave the server. Security is built in at every layer and each control is backed by an automated test, including integration tests against the real Firebase emulators. Privacy is a design rule, not an option: nothing a user captures or asks is stored or logged. Performance work brought the shortcut-to-crop time to roughly a quarter to a half second and cut the desktop footprint from about 397 MB to about 183 MB.

References

Bibliography

[1] Firebase Authentication and the Admin SDK (`verifyIdToken`). `https://firebase.google.com/docs/auth`

[2] Cloud Firestore transactions and security rules. `https://firebase.google.com/docs/firestore`

[3] Firebase Local Emulator Suite. `https://firebase.google.com/docs/emulator-suite`

[4] Google Cloud Run documentation. `https://cloud.google.com/run/docs`

[5] Secret Manager. `https://cloud.google.com/secret-manager/docs`

[6] Render documentation, including Blueprint specification. `https://render.com/docs`

[7] Gemini API: streaming, image input, Google Search grounding. `https://ai.google.dev/gemini-api/docs`

[8] Electron documentation: security checklist, performance, `safeStorage`. `https://www.electronjs.org/docs/latest`

[9] OWASP API Security Top 10 (2023). `https://owasp.org/API-Security/`

[10] Tesseract.js. `https://github.com/naptha/tesseract.js`

[11] Lucide icons. `https://lucide.dev`